PHASE 4 · CI/CD SUPPLY CHAIN · ADVANCED

Chapter 16

# DefectDojo

## Vulnerability Management & Remediation

Centralize findings from SonarQube and Harbor/Trivy into a single pane of glass — risk scoring, SLA tracking, and audit-ready compliance reporting.

*Estimated time: 3–4 hours*

# Why DefectDojo Matters

Vulnerabilities surface from many independent tools across the pipeline. Without a unifying layer, findings stay scattered, duplicated, and impossible to prioritize.

| | | |
|---|---|---|
| **Code** | **SonarQube** | SAST — bugs, injection flaws |
| **Dependencies** | **SonarQube / Snyk** | Vulnerable libraries |
| **Container Images** | **Harbor / Trivy** | OS package vulnerabilities |
| **Infrastructure** | **Various** | Misconfiguration |
| **API Security** | **DAST tools** | Runtime vulnerabilities |

# Docker Compose Stack

Deployed on sec-appsec-01 (same VM as SonarQube). Three containers behind a shared bridge network.




**Key setup steps**

1. Create directory structure → 2. Write docker-compose.yml → 3. Generate SECRET_KEY, run docker-compose up -d → 4. Create superuser via manage.py

# Stand Up the Org Structure

1

### Access the Web UI

https://10.0.2.15:8000 — sign in as admin

2

### Create Organization

Administration → Organizations
Name: "DevSecOps Lab"

3

### Create Team

Administration → Teams
Name: "Security", linked to the org

4

### Create Product

Products → Add Product
Name: "MyApp", Team: Security

*Credentials used in the lab: admin / DefectDojo123!  (change before any non-lab use)*

# SonarQube → DefectDojo

### Export from SonarQube

Projects → myapp → Issues
Export → JSON format

### Upload Results

Products → MyApp →
Upload Results, Scanner: SonarQube

### Findings Imported

Vulnerabilities, bugs, and
code smells all land in DefectDojo

CRITICAL | HIGH | MAJOR | MINOR / CODE SMELL

# Harbor / Trivy → DefectDojo

## Configure the Scanner

- Tools → Scanners → Add Scanner
- Name: Harbor Trivy
- Scanner type: Trivy
- URI: https://harbor.devsecops.com/api/v2.0
- Product → MyApp → API Settings → attach scanner credentials

## Automatic Polling

- Administration → System Settings
- Enable Harbor integration
- Polling interval: 1 hour
- Auto-import findings: enabled

---

**Automatically imports:**

- Container image vulnerabilities (CVEs)
- OS package vulnerabilities
- Dependency vulnerabilities

# Risk Scoring

**Risk Score = Severity × Probability × Impact**

| | |
|---|---|
| CRITICAL | 9.0 + |
| HIGH | 7.0 – 8.9 |
| MEDIUM | 4.0 – 6.9 |
| LOW | 1.0 – 3.9 |

## Example Findings

| | | |
|---|---|---|
| SQL Injection | CRITICAL | 9.5 |
| Cross-Site Scripting (XSS) | HIGH | 7.8 |
| Log Injection | MEDIUM | 5.2 |

# Remediation Tracking & SLA

## Remediation Workflow

1. View Finding
2. Assign to Developer
3. Set Due Date
4. Track Status




### SLA Targets

| | |
|---|---|
| Critical | 24 hours |
| High | 7 days |
| Medium | 30 days |
| Low | 90 days |

**DefectDojo tracks:**

- Days since discovery
- Days remaining per SLA
- Breaches (overdue findings)

# Executive & Compliance Reporting

### MyApp Vulnerability Report — Q1 2025

Total Findings: 47

| 2 | 8 | 22 | 15 |
|---|---|---|---|
| CRITICAL | HIGH | MEDIUM | LOW |

**On Time: 45** **Overdue (SLA Breach): 2**

Top Vulnerability Type: SQL Injection (6 instances)

Top Vulnerable Component: Django 3.1 (8 findings)

### Compliance Reports Generated For Auditors

- OWASP Top 10 (2021)
- CWE — Common Weakness Enumeration
- CVSS Scores
- PCI DSS Compliance
- HIPAA Compliance

# Gate the Pipeline on Critical Findings

## Fail Pipeline on Critical Vulns

```
defectdojo_check:
  stage: scan
  script:
    - CRITICAL=$(curl -s
      -H "Authorization: Token $TOKEN"
      .../api/v2/findings/?severity=Critical
      | jq '.count')
    - if [ "$CRITICAL" -gt 0 ]; then
        echo "Critical vulns found"; exit 1;
      fi
  only:
    - merge_requests
```

## Auto-Create Issues for Findings

- Administration → Integrations → GitLab
- Enable: create GitLab issues for findings
- Severity threshold: HIGH
- Auto-assign to: security-team

**Title:** "SQL Injection in user_auth.py:45"
**Assignee:** Security team lead
**Labels:** security, critical

16.9 — ADVANCED

# DAST Integration

Dynamic Application Security Testing — runtime vulnerability discovery against a live staging environment.




*Administration → Tools → Scanners → Add each scanner*

### Configure for staging environment

- Tools → Schedule Scans
- DAST (ZAP): Nightly on staging
- Import findings: 6 AM

# Trending Toward Fewer, Faster Fixes

### Vulnerability Density

Findings per KLOC (1000 lines of code)

### Mean Time to Remediate

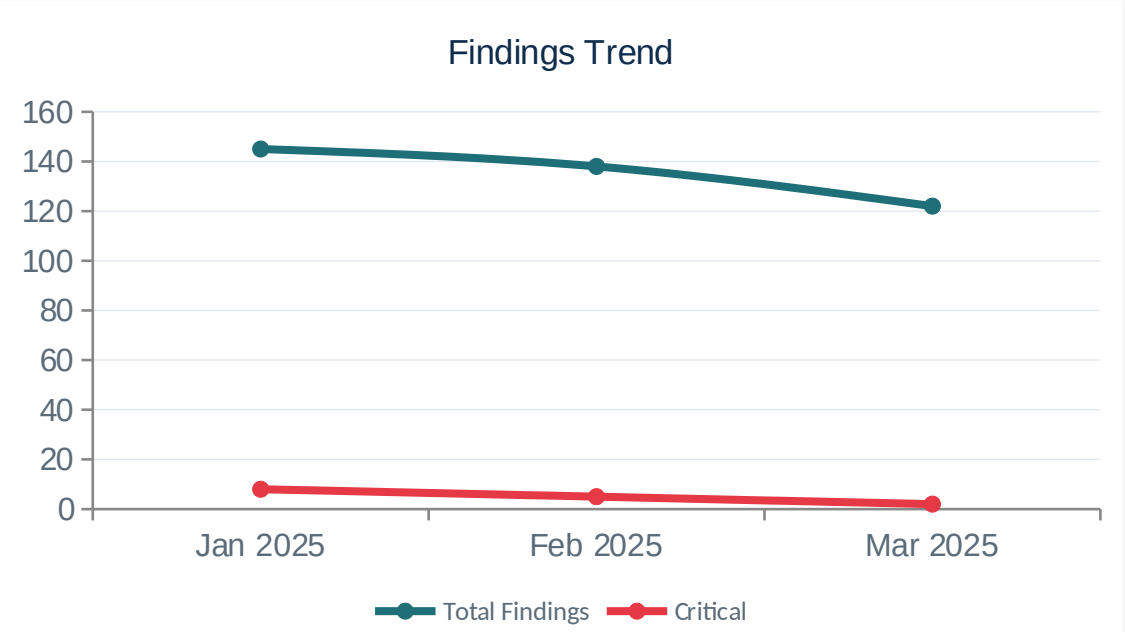
Average days to fix

### SLA Compliance

% of findings fixed on time

### Critical Backlog

# of overdue critical findings




**↓ Trend: IMPROVING — fewer findings, faster fixes (MTTR 12d → 9d, SLA 92% → 98%)**

# Confirming DefectDojo Is Operational

```
#!/bin/bash
# ~/lab/scripts/validate-chapter-16.sh
set -euo pipefail

DEFECTDOJO_IP="10.0.2.15"

# --- DefectDojo Service ---
ssh sec-appsec-01 'docker ps | grep -q defectdojo'
  ✅ PASS: DefectDojo container running

# --- Web UI Access ---
curl -s -k https://$DEFECTDOJO_IP:8000 | grep -qi defectdojo
  ✅ PASS: DefectDojo web UI accessible

# --- Results ---
  Passed: 2   Failed: 0
  🎉 DefectDojo is operational!
```

### What the script checks

- Container is running on sec-appsec-01
- Web UI responds on port 8000
- Response body confirms DefectDojo branding

---

*Run after every install/config change to confirm the stack came back up cleanly.*

# What We Built

| | |
|---|---|
| **DefectDojo server** | Centralized vulnerability management |
| **SonarQube integration** | Import SAST findings |
| **Harbor integration** | Import image scan findings |
| **Risk scoring** | Prioritize remediation |
| **SLA tracking** | Enforce time limits |
| **Reporting** | Compliance & audit reports |
| **Metric tracking** | Trending and improvement |

KEY TAKEAWAY

# Vulnerabilities Are Managed, Not Buried

### BEFORE DEFECTDOJO

- Vulnerabilities scattered across tools
- Hard to prioritize
- No SLA tracking
- No executive visibility

### AFTER DEFECTDOJO

- **All vulnerabilities centralized**
- **Risk scoring prioritizes fix order**
- **SLA enforcement (time limits)**
- **Executive dashboards (visibility)**
- **Compliance reporting (audit ready)**